Mercury Project

Test Plan and Test Report

Test strategy, implemented coverage, release evidence, and acceptance criteria.

Course	CSE499 Senior Project
Author	Matthew D. Barker
Version	RC 1.2 (1.2.0-rc.1)
Date	July 2026
Use	Academic review and grading only

Document Purpose

This document defines the Mercury RC 1.2 test strategy and records the automated and manual evidence used to validate framework behavior, security claims, plugin boundaries, and release acceptance.

Test Objectives

- Prove valid protected exchanges recover the original payload.
- Prove AES-GCM and ChaCha20-Poly1305 satisfy the same provider contract.
- Prove Binary and JSON codecs preserve complete SecureEnvelope data.
- Prove tampered payload and authenticated envelope fields are rejected.
- Prove replayed protected frames are rejected after successful authentication.
- Prove wrong-key, missing-key, invalid-key-length, malformed-frame, and empty-input paths fail safely.
- Prove explicit client identity rejects unintended recipients.
- Prove transports preserve frames, handle concurrency, and enforce resource limits.
- Prove replay storage expiration, concurrency, and bounded capacity.
- Prove binary, large, and chunked payload scenarios work.
- Prove the demos visibly demonstrate the required security behavior.

Test Environment

Item	Value
Operating system	Windows development environment for the full suite and WinForms demonstration; console host also supports compatible non-Windows environments.
IDE	Visual Studio 2022 or later.
Framework target	.NET 10 for Mercury.Tests and current demo targets; framework libraries retain their defined compatibility targets.
Test framework	xUnit.
Primary providers	AesGcmCryptoProvider and ChaCha20CryptoProvider.
Pending provider	AES-CCM; two tests intentionally skipped.
Primary codecs	Binary and JSON.
Primary transports	In-memory, framed TCP, FileDrop, and test doubles.
Release candidate	Mercury RC 1.2 / 1.2.0-rc.1.

Test Strategy

Test Level	Purpose	Examples
Primitive and shared component tests	Validate value types, defensive copying, metadata behavior, key-provider rules, and utility contracts.	KeyId, AlgorithmId, ReadOnlyMemory, Metadata, SymmetricKeyProviderDictionary
Provider contract tests	Run common success and failure behavior against each completed crypto provider.	Round trip, fresh nonce/output, wrong key, invalid key length, payload tamper, replay token, algorithms, cancellation
Codec tests	Validate Binary and JSON round trips and malformed-input rejection.	Full envelope preservation, empty input, wrong codec, malformed JSON, truncated Binary

Test Level	Purpose	Examples
Replay tests	Validate duplicate rejection, sender scoping, concurrency, expiration, capacity, and cancellation.	InMemoryReplayProtectorTests
Transport tests	Validate frame delivery, concurrency, cancellation, size limits, and transport-specific behavior.	In-memory, TCP, FileDrop
Pipeline tests	Validate integrated send/receive behavior using provider/codec/transport combinations.	Round trip, binary payload, large payload, wrong key, recipient mismatch, replay, tamper
Failure-path tests	Verify controlled failures, no plaintext release, and provider/replay call ordering.	Decode failure, provider failure, authentication failure, replay failure, transport failure
EasySetup tests	Validate simplified construction without weakening core boundaries.	Factory and client creation tests
Manual demo tests	Validate user-visible release behavior.	WinForms and console success, replay, tamper, wrong key, TCP, file transfer

Automated Test Coverage

ID	Test	Scenario	Expected Result	Status
TC-001	Provider round trip	Valid payload through each completed provider.	Original payload recovered exactly.	Implemented
TC-002	Provider freshness	Same payload protected twice.	Protected payload, replay token, and envelope ID differ.	Implemented
TC-003	Wrong key	Open a valid envelope using different key material.	AuthenticationFailed; no plaintext.	Implemented
TC-004	Invalid key length	Use empty, short, or long symmetric keys.	Controlled provider failure.	Implemented
TC-005	Payload tamper	Modify nonce, tag, or ciphertext bytes.	AuthenticationFailed; no plaintext.	Implemented
TC-006	Authenticated envelope tamper	Change envelope ID, timestamp, signature, header metadata, or footer metadata.	AuthenticationFailed across provider/codec combinations.	Implemented
TC-007	Sender/recipient/algorithm tamper	Change authenticated identity or algorithm fields.	AuthenticationFailed; no plaintext.	Implemented
TC-008	Replay token tamper	Change or remove replay token.	Authentication or structural failure; no plaintext.	Implemented
TC-009	Codec round trip	Encode and decode a complete SecureEnvelope through Binary and JSON.	All fields and protected payload preserved.	Implemented
TC-010	Malformed codec input	Empty, malformed, truncated, wrong-codec, or invalid frames.	Decode rejected before plaintext or provider processing.	Implemented
TC-011	Replay duplicate	Deliver same authenticated frame twice.	First accepted; duplicate rejected.	Implemented
TC-012	Replay concurrency	Submit duplicate token concurrently.	Exactly one accepted.	Implemented
TC-013	Replay expiration	Wait beyond configured replay window.	Expired record can be accepted again.	Implemented
TC-014	Replay capacity	Fill bounded replay store.	Additional unique token rejected; capacity returns after expiration.	Implemented
TC-015	Explicit recipient identity	Deliver an envelope to the wrong client identity.	Authentication/recipient failure; no plaintext.	Implemented
TC-016	Binary payload	Send arbitrary byte data including null and high-bit values.	Exact bytes recovered.	Implemented
TC-017	Large payload	Send a one-megabyte payload through supported pipeline combinations.	Exact payload recovered.	Implemented
TC-018	Transport frame limit	Send or declare an oversized frame.	Rejected before unbounded processing.	Implemented
TC-019	TCP frame round trip	Exchange protected frames over loopback TCP.	Same frame and original payload recovered.	Implemented

ID	Test	Scenario	Expected Result	Status
TC-020	FileDrop pipeline	Exchange frames through FileDrop transport.	Protected payload recovered for supported scenarios.	Implemented
TC-021	Cancellation	Use a canceled token in provider, client, replay, codec-supporting, and transport paths.	OperationCanceledException propagates where specified.	Implemented
TC-022	No plaintext on failure	Exercise decode, authentication, replay, provider, and transport failures.	Payload remains empty.	Implemented
TC-023	AES-CCM provider	Run provider and pipeline contract tests.	Pending provider passes common contract.	Skipped - pending

Provider and Codec Matrix

Provider	Binary Codec	JSON Codec	Status
AES-GCM	Contract, pipeline, tamper, replay, and failure tests	Contract, pipeline, tamper, replay, and failure tests	Passed
ChaCha20-Poly1305	Contract, pipeline, tamper, replay, and failure tests	Contract, pipeline, tamper, replay, and failure tests	Passed
AES-CCM	Pending	Pending	Two tests skipped

Manual Demo Test Cases

Demo	Steps	Expected Result	Status
Secure round trip	Configure provider, codec, and transport; send payload.	Recovered payload matches original and security status remains healthy.	Verified
Replay attack	Capture a valid protected frame and resend it.	Duplicate rejected; no plaintext released.	Verified
Tamper detection	Modify protected payload in the TCP attack simulator.	Authentication/integrity fails; receive blocked.	Verified
Wrong key	Use mismatched receiving key material.	Authentication fails; no plaintext released.	Verified
TCP transport	Send a protected frame over loopback framed TCP.	Frame delivered and original payload recovered when not attacked.	Verified
Binary and JSON codecs	Run normal and tamper scenarios with each codec.	Valid frames succeed; modified frames are rejected.	Verified
Secure file transfer	Send a file using chunking where appropriate.	Recovered data matches source after successful validation.	Verified
Console host	Run cross-platform menu scenarios.	Provider, codec, payload, and file scenarios complete successfully.	Verified

Release Test Result

Date	Build / Version	Environment	Result	Notes
July 2026	Mercury RC 1.2 / 1.2.0-rc.1	Windows developer environment, Visual Studio Test Explorer	330 discovered; 328 passed; 2 skipped; 0 failed	Only the two pending AES-CCM tests are skipped.
July 2026	Mercury RC 1.2 demos	WinForms and console demonstration environments	Pass	Normal exchange and security demonstrations verified.

Acceptance Criteria

1. The solution builds from a clean checkout using the documented SDK.
2. The full automated suite completes with zero failed tests.
3. Only the two documented pending AES-CCM tests may remain skipped for RC 1.2.
4. AES-GCM and ChaCha20-Poly1305 pass their shared contract and pipeline tests.
5. Binary and JSON codecs pass round-trip and malformed-input tests.
6. Tampered communications do not release plaintext.
7. Replay attempts do not release plaintext.
8. Wrong-key and wrong-recipient attempts do not release plaintext.
9. The WinForms and console demos run without unhandled exceptions in documented scenarios.
10. Release evidence is recorded with version, date, environment, and pass/fail status.

Traceability Summary

The implemented test suite covers the functional and nonfunctional requirements for protected send and receive, confidentiality, tamper detection, authenticated envelope data, replay rejection, wrong-key failure, provider/codec/transport abstractions, metadata, binary payloads, explicit client identity, bounded replay storage, frame limits, cancellation, and safe failure. AES-CCM remains the only documented pending provider item in RC 1.2.